ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА

ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №4

«Професійне ООП та система типів Python»

з курсу «Професійний Python»
Варіант №1 — «Система обліку студентів»

Виконала: Руссу Анастасія Сергіївна

Група: ФЕП-32

Перевірив: Куньо І. М.

Дата: 26.10.2026

Львів — 2026

1. Мета роботи

Побудувати типізовану об’єктну модель системи обліку студентів. Використати композицію, наслідування, абстрактні класи, протоколи, dataclass, properties та спеціальні методи. Організувати залежності за принципами SOLID і перевірити програму за допомогою mypy.

2. Індивідуальний варіант і постановка задачі

Варіант №1 — «Система обліку студентів». Програма повинна додавати й редагувати студентів, шукати за ID, прізвищем і групою, обчислювати середній бал та визначати найкращого студента. Окремо потрібні repository, reporting service та абстракція повідомлень.

Student представляє студента, Group — академічну групу, AcademicRecord — набір його оцінок, Grade — одну оцінку, StudentRegistry — колекцію студентів із пошуком і статистикою. Дані передаються у сервіс як StudentPayload, а потім перетворюються на об’єкти предметної області.

У демонстрації використано шість вигаданих студентів із попередніх робіт. Кожен має дві оцінки. Середній бал студента обчислюється за його оцінками, а загальний — як середнє індивідуальних середніх. Студенти без оцінок не беруть участі в цьому обчисленні. Для порожньої вибірки результат дорівнює None. За однакових максимальних балів обирається перший доданий студент.

Оцінки належать діапазону 0–100. Значення nan та inf відхиляються. Порожні імена й назви груп не допускаються. Повторний ID спричиняє ValueError. Пошук за прізвищем не залежить від регістру, а група визначається її ID та назвою.

3. Структура проєкту

```
Lab_04_variant_1/
├── pyproject.toml
├── README.md
├── .gitignore
├── src/domain_app/
│   ├── __init__.py
│   ├── py.typed
│   ├── value_objects.py
│   ├── models.py
│   ├── protocols.py
│   ├── repositories.py
│   ├── dto.py
│   ├── reporting.py
│   ├── services.py
│   ├── main.py
│   ├── experiments.py
│   └── type_experiment.py
└── tests/test_domain.py
```

value_objects містить Grade і Group; models — Student, AcademicRecord і StudentRegistry. Протоколи та репозиторій винесено окремо. services реалізує прикладні операції, reporting — звіти. main демонструє роботу, experiments порівнює архітектурні підходи, type_experiment показує помилки типізації та їх виправлення.

4. Опис створених модулів

За основу взято приклади dataclass, контейнера, ABC, Protocol, generic repository і dependency injection з методички. Приклад інтернет-магазину адаптовано до варіанта обліку студентів. Публічні методи мають анотації параметрів і результатів.

4.1. Діаграма класів

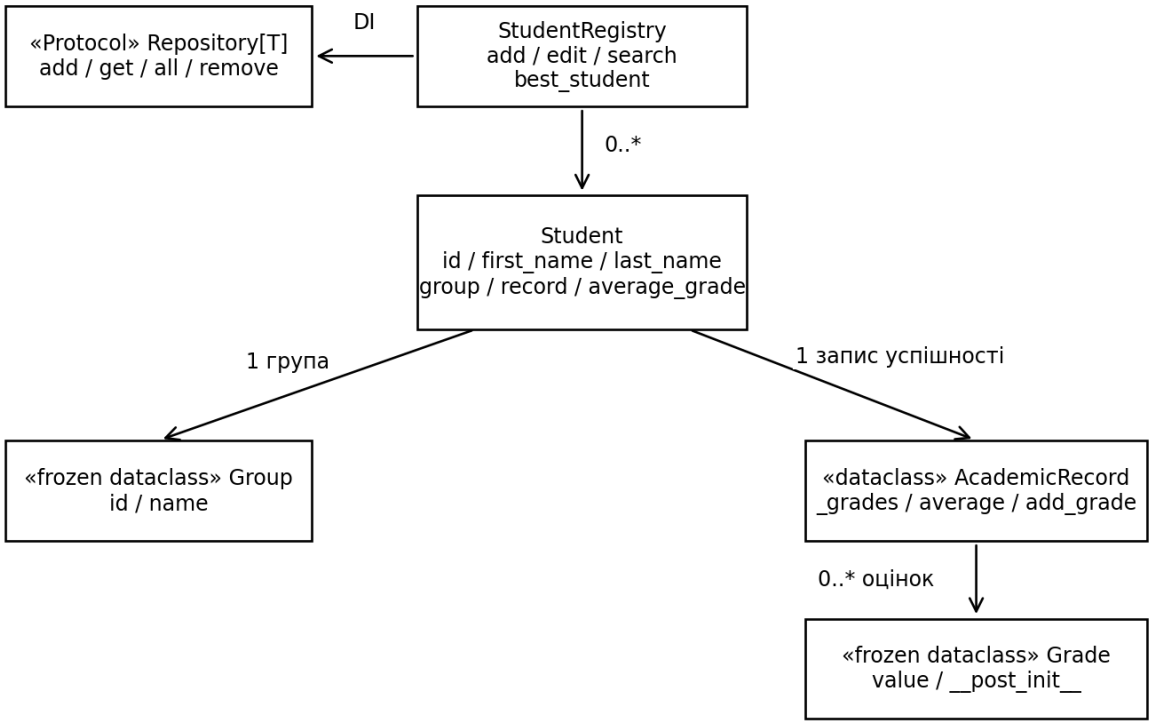


Рис. 1. Основні класи системи обліку студентів та їхні зв’язки

StudentRegistry працює через Repository[Student]. Студент посилається на одну Group і створює власний AcademicRecord, який містить оцінки Grade. Група може бути спільною для кількох студентів, а запис успішності належить конкретному студентові. На схемі стрілки позначають напрям використання об’єктів, а числа — кількість пов’язаних об’єктів.

Наслідування застосовано для форматів звіту: TableReport і BriefReport є реалізаціями StudentReport. ReportingService використовує об’єкт формату через композицію. StudentService отримує StudentRegistry та Notifier, а конкретний спосіб повідомлення визначається під час створення сервісу.

4.2. value_objects.py

```
from dataclasses import dataclass
from math import isfinite
@dataclass(frozen=True, slots=True)
class Grade:
    value: float
    def __post_init__(self) -> None:
        if isinstance(self.value, bool) or not isfinite(self.value):
            raise ValueError("Бал має бути скінченним числом.")
        if not 0 <= self.value <= 100:
            raise ValueError("Бал має належати діапазону від 0 до 100.")
    def __str__(self) -> str:
        return f"{self.value:.2f}"
@dataclass(frozen=True, slots=True)
class Group:
    id: int
```

```
    name: str
    def __post_init__(self) -> None:
        if isinstance(self.id, bool) or self.id <= 0:
            raise ValueError("ID групи має бути додатним.")
        if not self.name.strip():
            raise ValueError("Назва групи не може бути порожньою.")
        object.__setattr__(self, "name", self.name.strip())
    def __str__(self) -> str:
        return self.name
```

Grade є immutable value object. Перевірка в __post_init__ відхиляє некоректні оцінки ще під час створення. frozen=True забороняє звичайне присвоєння полям, slots=True обмежує набір атрибутів. Group також є dataclass; крайні пробіли в назві прибираються під час ініціалізації.

4.3. AcademicRecord

```
@dataclass
class AcademicRecord:
    _grades: list[Grade] = field(default_factory=list, init=False, repr=False)
    def add_grade(self, grade: Grade) -> None:
        self._grades.append(grade)
    @property
    def grades(self) -> tuple[Grade, ...]:
        return tuple(self._grades)
    @property
    def average(self) -> float | None:
        if not self._grades:
            return None
        return sum(grade.value for grade in self._grades) / len(self._grades)
    def __len__(self) -> int:
        return len(self._grades)
    def __iter__(self) -> Iterator[Grade]:
        return iter(self._grades)
```

default_factory створює окремий список оцінок для кожного об’єкта. Оцінки додаються методом add_grade. Властивість grades повертає tuple, тому зовнішній код не отримує змінюваний список. average обчислює середнє, __len__ повертає кількість оцінок, __iter__ дозволяє перебирати їх у циклі.

4.4. Student і контроль стану

ID доступний лише для читання, щоб зміна ідентифікатора не порушила ключі репозиторію. Сеттери first_name і last_name перевіряють непорожність та прибирають крайні пробіли. Студент сам створює AcademicRecord; це приклад композиції. Префікс _ позначає внутрішні атрибути, але не є абсолютним обмеженням доступу у Python.

```
class Student:
    def __init__(self, student_id: int, first_name: str,
                 last_name: str, group: Group) -> None:
        if isinstance(student_id, bool) or student_id <= 0:
            raise ValueError("ID студента має бути додатним.")
        self._id = student_id
        self.first_name = first_name
        self.last_name = last_name
        self.group = group
        self._record = AcademicRecord()
    @property
    def id(self) -> int:
        return self._id
```

```
    @property
    def first_name(self) -> str:
        return self._first_name
    @first_name.setter
    def first_name(self, value: str) -> None:
        if not value.strip():
            raise ValueError("Ім'я не може бути порожнім.")
        self._first_name = value.strip()
    @property
    def last_name(self) -> str:
        return self._last_name
    @last_name.setter
    def last_name(self, value: str) -> None:
        if not value.strip():
            raise ValueError("Прізвище не може бути порожнім.")
        self._last_name = value.strip()
    @property
    def record(self) -> AcademicRecord:
        return self._record
    @property
    def average_grade(self) -> float | None:
        return self.record.average
    def __str__(self) -> str:
        return f"{self.last_name} {self.first_name} ({self.group})"
    def __repr__(self) -> str:
        return (f"Student(student_id={self.id!r}, first_name={self.first_name!r}, "
                f"last_name={self.last_name!r}, group={self.group!r})")
```

4.5. StudentRegistry

```
class StudentRegistry:
    def __init__(self, repository: Repository[Student]) -> None:
        self._repository = repository
    def add(self, student: Student) -> None:
        self._repository.add(student)
    def get(self, student_id: int) -> Student | None:
        return self._repository.get(student_id)
    def edit(self, student_id: int, *, first_name: str | None = None,
             last_name: str | None = None, group: Group | None = None) -> Student:
        student = self.get(student_id)
        if student is None:
            raise KeyError(f"Студента з ID {student_id} не знайдено.")
        if first_name is not None and not first_name.strip():
            raise ValueError("Ім'я не може бути порожнім.")
        if last_name is not None and not last_name.strip():
            raise ValueError("Прізвище не може бути порожнім.")
        if first_name is not None:
            student.first_name = first_name
        if last_name is not None:
            student.last_name = last_name
        if group is not None:
            student.group = group
        return student
    def search(self, last_name: str | None = None,
               group: Group | None = None) -> list[Student]:
        return [student for student in self
                if (last_name is None or student.last_name.casefold()
                    == last_name.strip().casefold())
                and (group is None or student.group == group)]
    def average_grade(self) -> float | None:
        grades = [student.average_grade for student in self
```

```
                if student.average_grade is not None]
        return sum(grades) / len(grades) if grades else None
    def best_student(self) -> Student | None:
        best = None
        maximum = -1.0
        for student in self:
            average = student.average_grade
            if average is not None and average > maximum:
                best, maximum = student, average
        return best
    def __len__(self) -> int:
        return len(self._repository.all())
    def __iter__(self) -> Iterator[Student]:
        return iter(self._repository.all())
    def __contains__(self, student_id: int) -> bool:
        return self.get(student_id) is not None
    def __getitem__(self, student_id: int) -> Student:
        student = self.get(student_id)
        if student is None:
            raise KeyError(student_id)
        return student
```

Реєстр перевіряє обидва імені до зміни даних. get повертає None для невідомого ID, а registry[id] та редагування піднімають KeyError. Індекс означає ID студента.

4.6. protocols.py та repositories.py

```
from typing import Protocol, TypeVar, runtime_checkable
class HasId(Protocol):
    @property
    def id(self) -> int: ...
T = TypeVar("T", bound=HasId)
class Repository(Protocol[T]):
    def add(self, item: T) -> None: ...
    def get(self, item_id: int) -> T | None: ...
    def all(self) -> list[T]: ...
    def remove(self, item_id: int) -> bool: ...
@runtime_checkable
class Notifier(Protocol):
    def send(self, recipient: str, message: str) -> None: ...
```

```
from typing import Generic
from domain_app.protocols import T
class InMemoryRepository(Generic[T]):
    def __init__(self) -> None:
        self._items: dict[int, T] = {}
    def add(self, item: T) -> None:
        if item.id in self._items:
            raise ValueError(f"ID {item.id} вже існує.")
        self._items[item.id] = item
    def get(self, item_id: int) -> T | None:
        return self._items.get(item_id)
    def all(self) -> list[T]:
        return list(self._items.values())
    def remove(self, item_id: int) -> bool:
        return self._items.pop(item_id, None) is not None
```

TypeVar T обмежений протоколом HasId. Завдяки цьому repository може зберігати різні типи з властивістю id. У main створюються InMemoryRepository[Student] та InMemoryRepository[Group]. Repository[T] описує контракт сховища, а Notifier — окремий контракт повідомлення.

runtime_checkable дозволяє перевірити наявність інтерфейсу через isinstance, але не перевіряє повні сигнатури типів у runtime.

4.7. dto.py

```
from typing import TypedDict
from domain_app.models import Student
from domain_app.value_objects import Grade, Group
class StudentPayload(TypedDict):
    student_id: int
    first_name: str
    last_name: str
    group_id: int
    group: str
    grades: list[float]
def student_from_payload(data: StudentPayload) -> Student:
    student = Student(data["student_id"], data["first_name"], data["last_name"],
                      Group(data["group_id"], data["group"]))
    for value in data["grades"]:
        student.record.add_grade(Grade(value))
    return student
```

StudentPayload є TypedDict для словника вхідних даних. Він не створює перевірку типів автоматично під час виконання. Функція student_from_payload створює Student, Group і Grade; об’єкти перевіряють свої обмеження. Список grades перетворюється на власний запис успішності, тому зовнішній словник не є сховищем стану студента.

4.8. Наслідування та reporting service

```
from abc import ABC, abstractmethod
from collections.abc import Sequence
from domain_app.models import Student, StudentRegistry
class StudentReport(ABC):
    def __init__(self, title: str) -> None:
        self.title = title
    @abstractmethod
    def render(self, students: Sequence[Student]) -> str: ...
class TableReport(StudentReport):
    def __init__(self) -> None:
        super().__init__("РЕЄСТР СТУДЕНТІВ")
    def render(self, students: Sequence[Student]) -> str:
        lines = [self.title,
                 f"{'ID':<5} {'Прізвище та ім’я':<25} {'Група':<10} {'Середній бал':>12}"]
        for student in students:
            average = student.average_grade
            grade = "немає" if average is None else f"{average:.2f}"
            name = f"{student.last_name} {student.first_name}"
            lines.append(f"{student.id:<5} {name:<25} {student.group.name:<10} {grade:>12}")
        if not students:
            lines.append("Студентів немає.")
        return "\n".join(lines)
class BriefReport(StudentReport):
    def __init__(self) -> None:
        super().__init__("КОРОТКИЙ ЗВІТ")
    def render(self, students: Sequence[Student]) -> str:
        return f"{self.title}: студентів — {len(students)}"
class ReportingService:
    def __init__(self, registry: StudentRegistry, report: StudentReport) -> None:
        self._registry = registry
        self._report = report
```

```
    def create_report(self) -> str:
        return self._report.render(list(self._registry))
```

StudentReport задає абстрактний метод render. TableReport і BriefReport викликають super().__init__ та реалізують один контракт. ReportingService приймає будь-яку реалізацію цього ABC, тому формат можна замінити без зміни сервісу.

4.9. services.py та dependency injection

```
from dataclasses import dataclass, field
from domain_app.dto import StudentPayload, student_from_payload
from domain_app.models import Student, StudentRegistry
from domain_app.protocols import Notifier
from domain_app.value_objects import Grade, Group
class ConsoleNotifier:
    def send(self, recipient: str, message: str) -> None:
        print(f"Повідомлення для {recipient}: {message}")
@dataclass
class MemoryNotifier:
    messages: list[tuple[str, str]] = field(default_factory=list)
    def send(self, recipient: str, message: str) -> None:
        self.messages.append((recipient, message))
class StudentService:
    def __init__(self, registry: StudentRegistry, notifier: Notifier) -> None:
        self._registry = registry
        self._notifier = notifier
    def add_student(self, data: StudentPayload) -> Student:
        student = student_from_payload(data)
        self._registry.add(student)
        self._notifier.send(str(student), "Дані додано до реєстру.")
        return student
    def edit_student(self, student_id: int, *, first_name: str | None = None,
                     last_name: str | None = None, group: Group | None = None) -> Student:
        student = self._registry.edit(student_id, first_name=first_name,
                                      last_name=last_name, group=group)
        self._notifier.send(str(student), "Дані оновлено.")
        return student
    def add_grade(self, student_id: int, value: float) -> None:
        student = self._registry[student_id]
        student.record.add_grade(Grade(value))
        self._notifier.send(str(student), "Бал додано.")
```

StudentService організовує додавання, редагування й додавання бала, а після успіху викликає Notifier. ConsoleNotifier друкує повідомлення, MemoryNotifier записує їх у список. Обидва класи структурно відповідають Protocol без явного наслідування. Невдала валідація не додає запис і не створює повідомлення.

4.10. Демонстраційний сценарій main.py

```
from domain_app.dto import StudentPayload
from domain_app.models import Student, StudentRegistry
from domain_app.protocols import Notifier
from domain_app.reporting import BriefReport, ReportingService, TableReport
from domain_app.repositories import InMemoryRepository
from domain_app.services import ConsoleNotifier, StudentService
from domain_app.value_objects import Grade, Group
def main() -> None:
    groups = InMemoryRepository[Group]()
    for group in (Group(1, "ФЕП-32"), Group(2, "ФЕП-31"), Group(3, "ФЕП-33")):
        groups.add(group)
```

```
    registry = StudentRegistry(InMemoryRepository[Student]())
    notifier = ConsoleNotifier()
    service = StudentService(registry, notifier)
    payloads: list[StudentPayload] = [
        {"student_id": 1, "first_name": "Марія", "last_name": "Дорош",
         "group_id": 1, "group": "ФЕП-32", "grades": [80.0, 93.0]},
        {"student_id": 2, "first_name": "Данило", "last_name": "Левицький",
         "group_id": 2, "group": "ФЕП-31", "grades": [95.0, 95.0]},
        {"student_id": 3, "first_name": "Соломія", "last_name": "Климчук",
         "group_id": 1, "group": "ФЕП-32", "grades": [90.0, 93.0]},
        {"student_id": 4, "first_name": "Остап", "last_name": "Мазур",
         "group_id": 2, "group": "ФЕП-31", "grades": [76.0, 80.0]},
        {"student_id": 5, "first_name": "Олена", "last_name": "Романюк",
         "group_id": 3, "group": "ФЕП-33", "grades": [84.0, 86.0]},
        {"student_id": 6, "first_name": "Тарас", "last_name": "Дорош",
         "group_id": 3, "group": "ФЕП-33", "grades": [70.0, 74.0]},
    ]
    for data in payloads:
        service.add_student(data)
    print()
    print(ReportingService(registry, TableReport()).create_report())
    average = registry.average_grade()
    if average is not None:
        print(f"Середній бал студентів: {average:.4f}")
    print("Найкращий студент:", registry.best_student())
```

Початковий набір містить шість студентів. Для кожного додаються дві оцінки. Після формування реєстру друкується таблиця, середній бал і найкращий студент.

4.10. Демонстраційний сценарій main.py — продовження

```
print("\nПОШУК ЗА ID 2")
    print(registry.get(2))
    print("ПОШУК ЗА ПРІЗВИЩЕМ ДОРОШ")
    for student in registry.search(last_name="ДОРОШ"):
        print(student)
    print("ПОШУК ЗА ГРУПОЮ ФЕП-32")
    for student in registry.search(group=Group(1, "ФЕП-32")):
        print(student)
    print("\nРЕДАГУВАННЯ СТУДЕНТА З ID 6")
    service.edit_student(6, first_name="Тарас Іван", group=Group(1, "ФЕП-32"))
    service.add_grade(6, 90)
    print(registry[6], "середній бал:", registry[6].average_grade)
    print("\nПОЛІМОРФІЗМ ЗВІТІВ")
    print(ReportingService(registry, BriefReport()).create_report())
    print("\nDUNDER METHODS I PROTOCOL")
    print("len(registry):", len(registry))
    print("2 in registry:", 2 in registry)
    print("registry[2]:", registry[2])
    print("repr(registry[2]):", repr(registry[2]))
    print("Бали студента 2:", [str(grade) for grade in registry[2].record])
    print("Notifier:", isinstance(notifier, Notifier))
    print("Репозиторій груп:", [str(group) for group in groups.all()])
    print("\nПЕРЕВІРКА НЕКОРЕКТНИХ ДАНИХ")
    try:
        Grade(105)
    except ValueError as error:
        print(error)
    try:
        registry[1].first_name = " "
    except ValueError as error:
```

```
        print(error)
    try:
        service.add_student(payloads[0])
    except ValueError as error:
        print(error)
    print("Пошук відсутнього ID:", registry.get(999))
    empty = StudentRegistry(InMemoryRepository[Student]())
    print("Порожній реєстр:", empty.average_grade(), empty.best_student())
if __name__ == "__main__":
    main()
```

Далі виконуються пошук, зміна імені та групи, додавання оцінки, заміна формату звіту й виклики спеціальних методів. Наприкінці демонструються помилки валідації, повторний ID і порожній реєстр. Код сценарію є послідовним; взаємодія з реєстром проходить через класи й сервіси.

4.11. Принципи SOLID

SRP. Grade перевіряє бал, AcademicRecord обчислює середнє, repository зберігає об'єкти, а StudentReport форматує текст. Зміна формату звіту не потребує зміни обчислення середнього.

OCP. Новий формат додається як реалізація StudentReport. ReportingService продовжує викликати render без перевірок конкретного типу.

LSP. TableReport і BriefReport приймають Sequence[Student] і повертають str. Обидва підтримують порожню послідовність, тому їх можна підставляти в ReportingService.

ISP. Notifier має лише send, а Repository — операції сховища. Сервіс повідомлень не зобов'язаний реалізовувати пошук чи форматування.

DIP. StudentRegistry залежить від Repository[Student], StudentService — від Notifier, ReportingService — від StudentReport. Конкретні реалізації передаються з main.

Додаткові можливості з розділу 74 методички: generic repository, bound TypeVar, runtime_checkable Protocol, власна колекція з __getitem__, __iter__ та __contains__, абстракція repository, immutable value objects і стратегія форматування звіту.

4.12. Перевірка граничних випадків

Тести охоплюють межі 0 і 100, від'ємні та завеликі оцінки, nan та inf, незмінність Grade, порожні імена, окремі списки оцінок, повторні ID, відсутні записи, пошук, атомарність редагування, однакові максимальні бали, підміну Notifier та рівність результатів двох способів обробки CSV. Параметризований тест оцінок дає п'ять окремих випадків; разом — 18 перевірок.

5. Файл pyproject.toml

```
[build-system]
requires = ["setuptools>=70"]
build-backend = "setuptools.build_meta"
[project]
name = "student-domain-app"
version = "0.1.0"
description = "Typed student registry for laboratory work 4"
requires-python = ">=3.11"
[project.optional-dependencies]
dev = ["pytest>=8", "mypy>=1.10"]
[project.scripts]
student-registry = "domain_app.main:main"
[tool.setuptools.packages.find]
```

```
where = ["src"]
[tool.setuptools.package-data]
domain_app = ["py.typed"]
[tool.pytest.ini_options]
testpaths = ["tests"]
[tool.mypy]
python_version = "3.11"
strict = true
warn_unused_ignores = true
warn_return_any = true
mypy_path = "src"
```

6. Файл .gitignore

```
.venv/
__pycache__/
.pytest_cache/
.mypy_cache/
*.egg-info/
build/
dist/
.vscode/
```

7. Фрагмент README.md

```
# Система обліку студентів
Лабораторна робота №4 з курсу «Професійний Python», варіант №1.
Авторка: Руссу Анастасія Сергіївна, ФЕП-32.
Викладач: Свелеба С. А.
## Призначення
Програма додає та редагує студентів, шукає за ID, прізвищем і групою,
обчислює середній бал та знаходить найкращого студента.
Демонстраційні дані шести студентів задано в main.py.
Результати та повідомлення виводяться в терміналі.
```

README також містить структуру, правила валідації, особливості пошуку, пояснення SOLID і команди експериментів. Середовище, кеші mypy та pytest не потрапляють до Git.

8. Створення virtual environment та запуск

У PowerShell команди виконуються з кореневого каталогу проєкту. Використовується Python 3.11. Спочатку створюється .venv і встановлюється пакет із залежностями для розробки.

```
& "C:\Program Files\Python311\python.exe" -m venv .venv
.\.venv\Scripts\python.exe -m pip install -e ".[dev]"
.\.venv\Scripts\python.exe -m domain_app.main
.\.venv\Scripts\python.exe -m mypy src tests
.\.venv\Scripts\python.exe -m pytest -q
.\.venv\Scripts\python.exe -m domain_app.type_experiment
.\.venv\Scripts\python.exe -m domain_app.experiments
```

Параметр -e пов’язує пакет із кодом у src. Додаткова група [dev] установлює pytest і mypy. Активація середовища не потрібна, оскільки шлях до його інтерпретатора зазначений безпосередньо. Налаштування mypy вмикає strict і цільову версію Python 3.11.

9. Результат роботи програми

9.1. Реєстр і пошук

Для початкового набору індивідуальні середні становлять 86,5; 95; 91,5; 78; 85 і 72. Загальне середнє дорівнює 84,6667, найкращий студент — Данило Левицький. За прізвищем «Дорош» знайдено Марію й Тараса, за групою ФЕП-32 — Марію й Соломію.

```
PS C:\Users\SunValley2\Downloads\121 спец\Третій курс\П'ятий семестр\Професійний Python\Lab_04_variant_1> .\.venv\Scripts\python.exe -m domain_app.main
Повідомлення для Дорош Марія (ФЕП-32): Дані додано до реєстру.
Повідомлення для Левицький Данило (ФЕП-31): Дані додано до реєстру.
Повідомлення для Кличчук Соломія (ФЕП-32): Дані додано до реєстру.
Повідомлення для Мазур Остап (ФЕП-31): Дані додано до реєстру.
Повідомлення для Романюк Олена (ФЕП-33): Дані додано до реєстру.
Повідомлення для Дорош Тарас (ФЕП-33): Дані додано до реєстру.

РЕЄСТР СТУДЕНТІВ
ID    Прізвище та ім'я         Група      Середній бал
1     Дорош Марія              ФЕП-32           86.50
2     Левицький Данило         ФЕП-31           95.00
3     Кличчук Соломія          ФЕП-32           91.50
4     Мазур Остап              ФЕП-31           78.00
5     Романюк Олена            ФЕП-33           85.00
6     Дорош Тарас              ФЕП-33           72.00
Середній бал студентів: 84.6667
Найкращий студент: Левицький Данило (ФЕП-31)

ПОШУК ЗА ID 2
Левицький Данило (ФЕП-31)
ПОШУК ЗА ПРІЗВИЩЕМ ДОРОШ
Дорош Марія (ФЕП-32)
Дорош Тарас (ФЕП-33)
ПОШУК ЗА ГРУПОЮ ФЕП-32
Дорош Марія (ФЕП-32)
Кличчук Соломія (ФЕП-32)
```

Рис. 2. Додавання студентів, формування звіту та пошук

9.2. Редагування й спеціальні методи

Для студента з ID 6 змінюється ім'я та група, після додавання бала 90 середнє стає 78. Реєстр містить шість студентів. Перевірка 2 in registry повертає True. Некоректний бал, порожнє ім'я та повторний ID відхиляються.

```
РЕДАГУВАННЯ СТУДЕНТА З ID 6
Повідомлення для Дорош Тарас Іван (ФЕП-32): Дані оновлено.
Повідомлення для Дорош Тарас Іван (ФЕП-32): Бал додано.
Дорош Тарас Іван (ФЕП-32) середній бал: 78.0

ПОЛІМОРФІЗМ ЗВІТІВ
КОРОТКИЙ ЗВІТ: студентів — 6

DUNDER METHODS І PROTOCOL
len(registry): 6
2 in registry: True
registry[2]: Левицький Данило (ФЕП-31)
repr(registry[2]): Student(student_id=2, first_name='Данило', last_name='Левицький', group=Group(id=2, name='ФЕП-31'))
Бали студента 2: ['95.00', '95.00']
Notifier: True
Репозиторій груп: ['ФЕП-32', 'ФЕП-31', 'ФЕП-33']

ПЕРЕВІРКА НЕКОРЕКТНИХ ДАНИХ
Бал має належати діапазону від 0 до 100.
Ім'я не може бути порожнім.
ID 1 вже існує.
Пошук відсутнього ID: None
Порожній реєстр: None None
```

Рис. 3. Редагування студента, спеціальні методи та валідація

9.3. Статична перевірка типів

Для перевірки використовується mypy у strict-режимі. Команда перевіряє src та tests. Публічні API анотовані; TypedDict, Generic[T] і Protocol перевіряють сумісність передавання даних між модулями.

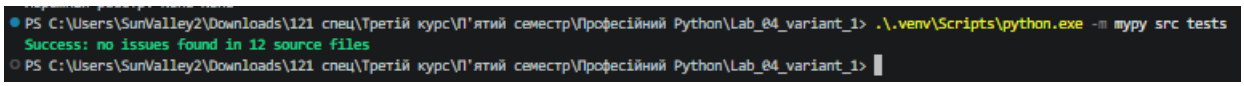


Рис. 4. Результат mypy для коду проєкту та тестів

9.4. Три навмисні помилки та виправлення

type_experiment записує тимчасовий type_demo.py, запускає mypy, виправляє цей файл і повторює перевірку. Файл автоматично видаляється після експерименту. Початковий варіант містить три помилки, передбачені розділом 71 методички.

```
from domain_app.models import Student
from domain_app.repositories import InMemoryRepository
from domain_app.value_objects import Group
student_id: int = "1"
def average() -> float:
    return "95.0"
repository = InMemoryRepository[Student]()
repository.add(Group(1, "ФЕП-32"))
```

У першому випадку змінній int присвоєно str — diagnostic [assignment]. У другому функція з результатом float повертає str — [return-value]. У третьому Repository[Student] отримує Group — [arg-type].

```
from domain_app.models import Student
from domain_app.repositories import InMemoryRepository
from domain_app.value_objects import Group
student_id: int = 1
def average() -> float:
    return 95.0
repository = InMemoryRepository[Student]()
repository.add(Student(student_id, "Марія", "Дорош", Group(1, "ФЕП-32")))
```

Виправлення полягають у використанні цілого ID, числового результату функції та об’єкта Student для відповідного repository. Після виправлення mypy не виявляє помилок цього файла.

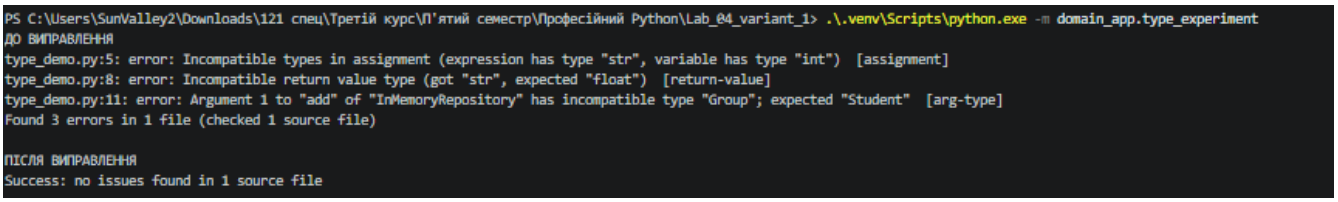


Рис. 5. Діагностика трьох помилок типів і повторна перевірка після виправлення

9.5. Створення залежності та dependency injection

```
class CoupledStudentService:
    def __init__(self, registry: StudentRegistry) -> None:
        self._registry = registry
        self._notifier = ConsoleNotifier()
    def add_student(self, data: StudentPayload) -> Student:
        student = student_from_payload(data)
        self._registry.add(student)
        self._notifier.send(str(student), "Дані додано до реєстру.")
        return student
```

У варіанті A сервіс сам створює ConsoleNotifier. Для тестування повідомлень потрібно перехоплювати консоль або змінювати залежність усередині об’єкта. У варіанті B StudentService

отримує Notifier у конструкторі; у тесті передається MemoryNotifier, а збережені повідомлення перевіряються звичайним assert.

Варіант B легше тестувати та розширювати. Він відповідає DIP, оскільки залежить від контракту Notifier. Підміна реалізації не змінює логіку додавання студента. Композиція з абстракцією знижує зв'язність між прикладною операцією та способом повідомлення.

9.6. Наслідування та композиція

```
class BaseProcessor(ABC):
    @abstractmethod
    def parse(self, text: str) -> list[StudentPayload]: ...
    def process(self, text: str) -> str:
        students = StudentValidator().validate(self.parse(text))
        return TableReport().render(students)
class CsvProcessor(BaseProcessor):
    def parse(self, text: str) -> list[StudentPayload]:
        return CsvParser().parse(text)
class Processor:
    def __init__(self, parser: CsvParser, validator: StudentValidator,
                 formatter: StudentReport) -> None:
        self._parser = parser
        self._validator = validator
        self._formatter = formatter
    def process(self, text: str) -> str:
        rows = self._parser.parse(text)
        students = self._validator.validate(rows)
        return self._formatter.render(students)
```

CsvProcessor успадковує алгоритм process від BaseProcessor і визначає parsing. Processor отримує parser, validator та formatter як окремі об'єкти. Для однакового CSV обидва способи повертають однакову таблицю. Заміна TableReport на BriefReport у композиції змінює формат без нового підкласу процесора.

9.6. Порівняння підходів — результати

Наслідування пов'язує процесор із базовим алгоритмом і зручне для стабільного шаблону обробки. Композиція дозволяє незалежно комбінувати компоненти, повторно використовувати їх та перевіряти окремо. У прикладі parser і validator мають конкретні типи, а formatter залежить від ABC; повне усунення залежностей від реалізацій потребувало б окремих протоколів для інших компонентів.

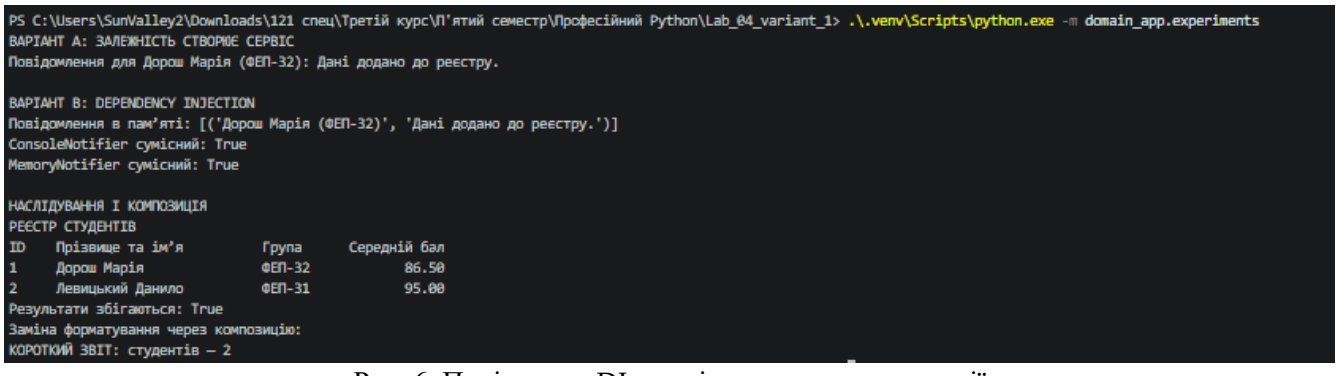


Рис. 6. Порівняння DI, наслідування та композиції

9.7. Автоматизовані тести

Команда pytest перевіряє 18 випадків. Окремо контролюється, що невдала операція не створює повідомлення й не змінює реєстр, а два підходи до обробки CSV дають однаковий результат.



Рис. 7. Результат виконання автоматизованих тестів

10. Робота з Git

Історія містить три послідовні етапи: налаштування пакета, реалізацію типізованої моделі та сервісів, додавання сценаріїв і тестів. Готовий звіт додається окремим комітом після оформлення скриншотів.

```
f9cde4a Configure typed Python package and development tools
67dc1a2 Implement typed student domain and injected services
6063192 Add demonstrations type checking experiments and tests
```

```
On branch main
nothing to commit, working tree clean
```

11. Відповіді на контрольні запитання

1. Що таке class?

Клас описує структуру й поведінку об’єктів певного типу.

2. Що таке object?

Об’єкт має власний стан, тип та ідентичність.

3. Що таке instance?

Instance — конкретний екземпляр класу, наприклад один Student.

4. Для чого __init__()?

Метод ініціалізує стан уже створеного екземпляра.

5. Що таке instance attribute?

Атрибут, значення якого належить окремому екземпляру.

6. Що таке class attribute?

Атрибут класу, доступний через клас та його екземпляри, якщо його не перекрито.

7. Що таке encapsulation?

Контроль внутрішнього стану через визначені операції та перевірки.

8. Для чого properties?

Для доступу до обчислюваних або контрольованих значень як до атрибутів.

9. Чим property відрізняється від public field?

Property може перевіряти чи обчислювати значення; public field зазвичай просто зберігає його.

10. Що таке inheritance?

Створення підкласу, який отримує та може уточнювати поведінку базового класу.

11. Що означає relation is-a?

Відношення «є різновидом». Наприклад, TableReport є StudentReport.

12. Що таке composition?

Побудова об’єкта з інших об’єктів, яким він делегує частину роботи.

13. Що означає relation has-a?

Відношення «має». Student має AcademicRecord.

14. Коли composition краща за inheritance?

Коли поведінку потрібно комбінувати й замінювати незалежними компонентами.

15. Для чого super()?

Для делегування виклику наступній реалізації за порядком пошуку методів MRO.

16. Що таке polymorphism?

Можливість використовувати різні реалізації через спільний інтерфейс.

17. Що таке duck typing?

Використання об’єкта за його доступною поведінкою, а не лише за конкретним класом.

18. Що таке ABC?

Абстрактний базовий клас, який описує спільний контракт і може містити реалізацію.

19. Для чого abstractmethod?

Позначає метод, який конкретний підклас повинен реалізувати.

20. Чи можна створити instance abstract class?

Ні, якщо в ньому залишилися нереалізовані абстрактні методи.

21. Що таке Protocol?

Опис структурного інтерфейсу для перевірки сумісності типів.

22. Чим Protocol відрізняється від ABC?

ABC зазвичай потребує належності до ієрархії; Protocol допускає сумісний клас без явного наслідування.

23. Що таке structural typing?

Сумісність визначається набором атрибутів і сигнатурами методів.

24. Що таке nominal typing?

Сумісність пов’язана з явно оголошеною належністю типу до ієрархії.

25. Що таке dataclass?

Декоратор класу, який генерує типові методи за оголошеними полями.

26. Які methods генерує dataclass?

Зазвичай __init__, __repr__ та __eq__; інші залежать від параметрів декоратора.

27. Для чого default_factory?

Щоб створювати окреме початкове значення для кожного екземпляра.

28. Чому mutable default є проблемою?

Спільний змінюваний об’єкт може спричинити зміну стану кількох екземплярів одночасно.

29. Для чого __post_init__()?

Для додаткових дій і перевірок після автоматично згенерованого __init__.

30. Що означає frozen=True?

Забороняє звичайне присвоєння та видалення полів; не робить вкладені змінювані об’єкти незмінними.

31. Що означає slots=True?

Генерує слоти для полів і зазвичай прибирає потребу в __dict__ екземпляра.

32. Що таке dunder method?

Спеціальний метод із подвійними підкресленнями для стандартних операцій Python.

33. Для чого __repr__()?

Для інформативного представлення об’єкта під час налагодження.

34. Для чого __str__()?

Для зручного текстового представлення об’єкта користувачеві.

35. Для чого __len__()?

Для визначення результату len(object).

36. Для чого __iter__()?

Для повернення ітератора й підтримки циклу for.

37. Для чого __contains__()?

Для перевірки входження через оператор in.

38. Для чого __call__()?

Щоб екземпляр можна було викликати як функцію.

39. Що таке SOLID?

П’ять принципів розподілу відповідальності та організації залежностей у програмі.

40. Що означає SRP?

Один клас має одну основну відповідальність і причину для зміни.

41. Що означає OCP?

Код відкритий для розширення поведінки без зміни стабільної реалізації.

42. Що означає LSP?

Підклас повинен замінювати базову абстракцію без порушення її контракту.

43. Що означає ISP?

Клієнт має залежати лише від потрібних йому частин інтерфейсу.

44. Що означає DIP?

Високорівнева логіка та реалізації залежать від абстракцій.

45. Що таке dependency injection?

Передавання готових залежностей ззовні, наприклад через конструктор.

46. Чому DI покращує testability?

Дозволяє передати тестову реалізацію замість зовнішнього сервісу.

47. Що таке type hint?

Анотація очікуваного типу змінної, параметра або результату.

48. Чи перевіряє Python type hints автоматично в runtime?

Ні. Для runtime-перевірок потрібні окремі умови або спеціальні засоби.

49. Що таке static type checker?

Інструмент аналізу типів без звичайного виконання перевірюваної програми.

50. Для чого mypy?

Для статичного пошуку несумісних типів та порушень контрактів API.

51. Для чого Pyright?

Для статичного аналізу типів у командному рядку та інтеграціях редактора.

52. Що таке TypeVar?

Змінна типу, яка пов’язує типи параметрів і результатів узагальненого коду.

53. Що таке Generic?

Засіб оголошення класів, параметризованих типами.

54. Що означає Generic[T]?

Клас використовує параметр типу T; Repository[Student] працює зі Student.

55. Що таке bound TypeVar?

TypeVar з верхньою межею сумісності, наприклад T, bound=HasId.

56. Що таке constrained TypeVar?

TypeVar із заданим набором допустимих типів, наприклад int і float.

57. Що таке TypedDict?

Опис словника з визначеними ключами та типами їхніх значень.

58. Чим TypedDict відрізняється від dataclass?

TypedDict у runtime залишається dict; dataclass створює екземпляри власного класу.

59. Коли доцільно використовувати TypedDict?

Для структурованих зовнішніх словників, JSON-подібних даних і payload.

60. Чому external payload і domain object краще розділяти?

Зовнішній формат може змінюватися незалежно від моделі. Перетворення дозволяє перевірити дані та зберегти правила предметної області.

12. Висновок

У ході лабораторної роботи розроблено типізовану систему обліку студентів. Сутності розділено за відповідальністю, оцінки подано незмінними об’єктами, а зміни даних проходять через перевірки. Композиція поєднує студента із записом успішності та сервіс із залежностями. Наслідування використано для форматів звіту. Generic repository повторно застосовується для студентів і груп. Експеримент із mypy показує помилки присвоєння, повернення та передавання аргументу, а DI дозволяє замінити консольні повідомлення тестовою реалізацією.